SKU Razor Engine: Complete System Documentation

1. System Overview

The **SKU Razor Engine** (also known as the Parser Engine) is a smart tool that converts messy and confusing supplier product titles into clean, standardized SKUs (Stock Keeping Units) for the mobile phone repair parts inventory.

Example:

- **Input Title:** `Fingerprint Sensor for Samsung Galaxy A52 A525 Black`
- **Output SKU:** `GALAXY A52 A525 FS BLACK`

How It Works: Hybrid Engine

The system uses a **hybrid approach** combining a strict rule engine with an AI fallback:

1. **Rule Engine:** Most product titles are processed instantly using our established rules and datasets. This is fast and highly accurate.
2. **AI Fallback:** If a title is completely new or too confusing for the rules, the system uses Artificial Intelligence to figure it out and generate the correct SKU.

---

2. System Pipeline

When a product title enters the system, it goes through a step-by-step pipeline to become a final SKU:

1. **Product Title:** The raw, messy text from the supplier.
2. **Text Normalization:** Converts everything to lowercase, removes extra spaces, and standardizes formatting.
3. **Noise Removal:** Strips out useless, non-identifying words (like "high quality", "100% original", "new").
4. **Model Detection:** Identifies the correct phone model (e.g., "Galaxy A52").
5. **Part Ontology Matching:** Finds the specific repair part and translates it to our standard acronym (e.g., "charging port" becomes "CP").
6. **Attribute Extraction:** Pulls out specific details like colors, sizes, or network types (e.g., "5G", "Black").
7. **Variant Detection:** Checks for special conditions or physical attributes (e.g., "with frame", "pulled").
8. **SKU Builder:** Assembles all the detected pieces into our standard, organized SKU format.
9. **31 Character Limit Validation:** Ensures the final SKU is not too long for the inventory management system.
10. **Duplicate Prevention:** Checks to make sure this exact SKU hasn't already been mapped to a fundamentally different product.
11. **Final SKU:** The clean, verified SKU is saved and used.

---

3. Current Training Datasets

The engine relies on specialized datasets to understand the language of mobile parts. Here is every dataset used and what it does:

1. Model Database

- **File Example:** `device_models.json`
- **Role:** Contains all supported phone models. The engine uses this to accurately recognize the device.
- **Examples:** `Galaxy A52`, `Galaxy S23 Ultra`, `Pixel 8 Pro`, `iPhone 13 Pro Max`

2. Model Code Database

- **File Example:** `model_codes.json`
- **Role:** Contains official manufacturer alphanumeric device codes. These are highly specific and improve detection accuracy dramatically.
- **Examples:** `A525`, `S918`, `G991`, `SM-A525F`

3. Part Ontology

- **File Example:** `part_ontology.json`
- **Role:** Defines the absolute standard abbreviations for our repair parts, ensuring consistent SKUs.
- **Examples:**
  - charging port → `CP`
  - fingerprint sensor → `FS`
  - earpiece speaker → `ES`
  - front camera → `FC`
  - back camera → `BC`
  - loud speaker → `LS`

4. Synonym Dataset

- **File Example:** `synonyms.json`
- **Role:** Maps unusual or varied phrasing from suppliers to our standard internal terms.
- **Examples:**
  - ear speaker → `earpiece speaker`
  - receiver speaker → `earpiece speaker`
  - charging socket → `charging port`

5. Spelling Correction Dataset

- **File Example:** `learned_spelling_variations.json`
- **Role:** Identifies and automatically corrects common typos made by suppliers.
- **Examples:**
  - spkeaker → `speaker`
  - battry → `battery`
  - galaxi → `galaxy`

6. Pattern Learning Dataset

- **File Example:** `learned_patterns.json`
- **Role:** Stores new phrase patterns the system previously discovered. This dataset grows automatically as the AI successfully parses new formats.
- **Examples:**
  - charging socket → `CP`
  - vibration motor → `VIB`

7. Color Dataset

- **File Example:** `color_dictionary.json`

- **Role:** Maps fancy, marketing-driven supplier color names to our standard, basic colors.
- **Examples:**
  - stormy black → `BLACK`
  - cloudy white → `WHITE`
  - midnight blue → `BLUE`

8. Variant Dataset

- **File Example:** `variant_attributes.json`
- **Role:** Stores special attributes or physical states of the part being parsed.
- **Examples:**
  - with bracket → `BRKT`
  - with frame → `FRAME`
  - with adhesive → `ADH`

9. Dataset Training Files

- **Examples:** Excel catalog imports, manual SKU corrections, inventory review feedback.
- **Role:** These external records are fed into the system by staff to continuously teach the engine new tricks, expanding the databases above.

---

4. Proposed Project File Structure

To keep the system organized, modular, and easy for any developer to maintain, the project structure should be organized as follows:

```
sku_engine/
│
├── data/
│   ├── models/
│   │   ├── device_models.json
│   │   └── model_codes.json
│   │
│   ├── ontology/
│   │   ├── part_ontology.json
│   │   ├── color_dictionary.json
│   │   └── variant_attributes.json
│   │
│   └── training/
│       ├── synonyms.json
│       ├── learned_patterns.json
│       └── learned_spelling_variations.json
│
├── datasets/
│   └── catalog_training_sets/  (Stores raw Excel files used for training)
│
└── docs/
    └── sku_engine_documentation.pdf
```

---

5. Data Validation Requirements & Data Quality Requirement

The core directive of the SKU Razor Engine is data integrity: **all training data entering the system must be clean and verified.** The engine acts as a strict gatekeeper against bad data.

Before any new data is learned or a final SKU is applied, it must pass these absolute validation rules:

1. **Unique Data:** The generated SKU must be unique; duplicate SKUs for different products are rejected.
2. **Strict Length Limit:** The SKU must be 31 characters or less to fit inventory database constraints.
3. **Capitalization:** The SKU must be entirely UPPERCASE.
4. **Valid Model Identity:** The detected phone model must perfectly exist in the Model Database. Invalid models cause rejection.
5. **Valid Part Code:** The part code (like CP or FS) must be known in the Part Ontology. Unknown parts cause rejection.
6. **Valid Color Base:** Any color used must map to a known entry in the Color Dictionary. Invalid colors cause rejection.

If a generated SKU or training data payload fails *any* of these checks, the Engine will **reject** the data. This guarantees that no incorrect, duplicate, or malformed SKUs ever exist in the active system.

---

6. Admin Training Dashboard

The inventory and administrative teams control the engine's intelligence using the **Admin Training Dashboard**. This allows non-developers to safely upgrade the engine's knowledge.

Through this panel, admins can:

- **Upload Datasets:** Bulk upload new supplier catalogs in Excel/CSV format to feed the engine fresh titles to parse.
- **Add Synonyms:** Teach the system that a new weird phrase (e.g., "charge slot") means something standard ("charging port").
- **Correct SKUs:** Manually override an incorrect SKU. When this happens, the system analyzes *why* it was wrong and learns the correction.
- **Add Spelling Corrections:** Directly input new, frequent typos suppliers are making so the engine ignores them next time.
- **Add Rule Patterns:** Create complex manual matching rules for specific brands or difficult title structures.

Every action taken in the dashboard directly updates the JSON datasets (Ontology, Spelling, Synonyms, etc.), making the rule engine permanently smarter without requiring a software developer to write code.